

Removing cache entries from Redis

Cache invalidation is one of the hard things. Sometimes we have incidents like #5478 where we end up with invalid data in the cache, and no convenient way to clear it. This document describes an approach for deleting a specified set of cache keys on GitLab.com without consuming excessive Redis resources.

This approach is split into two phases:

1. Obtain the keys to be deleted (against a secondary)
2. Delete the specified keys (against the primary)

By doing this, we gain a few benefits:

1. We perform as few operations on the primary as necessary. Secondaries do not receive application traffic, and so have more headroom and can tolerate being blocked briefly.
2. We can retain a list of deleted keys for later inspection - for instance, if a user reports an issue, we can check if their cache key was in the list.
3. Obtaining the keys to be deleted may be an iterative, exploratory process. We do not want to conflate this with data deletion, which should ideally be as simple as possible.

(In the future we may move to having version-based cache invalidation, where this will be simpler. One example is the way it's possible to invalidate the Markdown cache - which is in Postgres, not Redis - via the API.)

Step 1: obtain the keys to be deleted

On a **secondary**, we can run a Ruby script like the below. `REDIS_PASSWORD` is provided as an environment variable. In this example, we're looking for keys of the form `cache:gitlab:license_key*`, but the general approach will be the same no matter the pattern.

```
require 'redis'

pattern = "cache:gitlab:license_key*"
output = File.open('/tmp/keys-to-delete.txt', 'w')

redis = Redis.new(:url => "redis://#{ENV['REDIS_PASSWORD']}@127.0.0.1")
cursor = '0'

loop do
  cursor, keys = redis.scan(cursor, match: pattern, count: 100000)

  if keys.count > 0
    puts "Writing #{keys.count}"
    keys.each { |key| output.write(key + "\n") }
  end
end
```

```

    puts cursor
    break if cursor == '0'
end

```

Step 2: delete the keys

This can be run on the console node as `Gitlab::Redis::Cache` will connect to the primary by default. Make sure to copy `/tmp/keys-to-delete.txt` to the console node before starting. It sleeps for a second after every 10 000 deletions. In most cases there should not be many tens of thousands of deletions anyway.

```

lines = File.readlines('/tmp/keys-to-delete.txt')

count = 0
Gitlab::Redis::Cache.with do |redis|
  lines.each do |line|
    line.rstrip!

    # Gitlab::Redis::Cache automatically adds the cache:gitlab: namespace, so
    # we have to remove it. Otherwise we will try to delete keys of the form
    # cache:gitlab:cache:gitlab:...
    line.gsub!(/^cache:gitlab:/, '')

    redis.expire(line, 0)

    count += 1

    if (count % 10000 == 0)
      puts "count is #{count}, sleeping..."
      sleep 1
    end
  end
end
end

```